SQL COURSE: PRACTICAL SOLUTION MANUAL

১০০% কার্যকরী SQL কোড সমাধান নির্দেশিকা (মডিউল ০২ – ২০)

বিশুদ্ধ SQL স্টেটমেন্ট • SQLite 3.30+ স্ট্যান্ডার্ড • কোনো তাত্ত্বিক ভূমিকা ব্যতীত সরাসরি কুয়েরি

মডিউল 02: SELECT, alias ও DISTINCT

প্রশ্ন 1: customers টেবিল থেকে শুধুমাত্র গ্রাহকের নাম (full_name) এবং জেলা (city) কলাম দুটি প্রদর্শন করার SQL কুয়েরি লিখুন।

```
SELECT full_name, city
FROM customers;
```

প্রশ্ন 2: products টেবিল থেকে পণ্যের নাম এবং তার মূল্যের কলামের নাম পরিবর্তন করে 'Price_BDT' হিসেবে দেখানোর জন্য AS ব্যবহার করে কুয়েরি লিখুন।

```
SELECT product_name, price AS Price_BDT
FROM products;
```

প্রশ্ন 3: customers টেবিলে কোন কোন অনন্য (unique) জেলার গ্রাহক রয়েছেন, তা ডুপ্লিকেট বাদ দিয়ে দেখতে DISTINCT ব্যবহার করে কুয়েরি লিখুন।

```
SELECT DISTINCT city
FROM customers;
```

প্রশ্ন 4: products টেবিল থেকে পণ্যের বর্তমান মূল্যের সাথে অতিরিক্ত ৫০ টাকা ডেলিভারি চার্জ যোগ করে 'total_cost' কলাম তৈরি করে প্রদর্শন করার কুয়েরি লিখুন।

```
SELECT product_name, price + 50 AS total_cost
FROM products;
```

প্রশ্ন 5: customers টেবিল থেকে প্রতিটি গ্রাহকের নাম এবং জেলাকে স্ট্রিং কনক্যাটেনেশন (||) অপারেটর ব্যবহার করে 'Customer_Address' নামে একসাথে (যেমন: 'Md. Rafijur Rahman (Patuakhali)') প্রদর্শন করার কুয়েরি লিখুন।

```
SELECT full_name || ' (' || city || ')' AS Customer_Address
FROM customers;
```

মডিউল 03: WHERE দিয়ে সারি ফিল্টার করা

প্রশ্ন 1: products টেবিল থেকে যেসব পণ্যের একক মূল্য (price) ১,০০০ টাকার বেশি, সেগুলোর নাম ও মূল্য প্রদর্শন করার কুয়েরি লিখুন।

```
SELECT product_name, price
FROM products
WHERE price > 1000;
```

প্রশ্ন 2: customers টেবিল থেকে শুধুমাত্র বরিশাল (Barishal) জেলার সক্রিয় (status = 'Active') গ্রাহকদের তালিকা ফিল্টার করার কুয়েরি লিখুন।

```
SELECT *
FROM customers
WHERE city = 'Barishal' AND status = 'Active';
```

প্রশ্ন 3: customers টেবিল থেকে যেসব গ্রাহকের ফোন নম্বর সংরক্ষিত নেই (phone IS NULL), তাঁদের আইডি ও পুরো নাম বের করার কুয়েরি লিখুন।

```
SELECT customer_id, full_name
FROM customers
WHERE phone IS NULL;
```

প্রশ্ন 4: customers টেবিল থেকে যেসব গ্রাহকের ফোন নম্বর সংরক্ষিত আছে (phone IS NOT NULL), তাঁদের নাম ও ফোন নম্বর প্রদর্শন করার কুয়েরি লিখুন।

```
SELECT full_name, phone
FROM customers
WHERE phone IS NOT NULL;
```

প্রশ্ন 5: products টেবিল থেকে ৫০০ টাকার বেশি এবং ১৫০০ টাকার কম মূল্যের পণ্যগুলো বের করতে AND শর্তযুক্ত কুয়েরি লিখুন।

```
SELECT *
FROM products
WHERE price > 500 AND price < 1500;
```

মডিউল 04: IN, BETWEEN, LIKE, ORDER BY ও LIMIT

প্রশ্ন 1: IN অপারেটর ব্যবহার করে ভোলা (Bhola), বরগুনা (Barguna) এবং পটুয়াখালী (Patuakhali) জেলার গ্রাহকদের তালিকা প্রদর্শন করার কুয়েরি লিখুন।

```
SELECT *
FROM customers
WHERE city IN ('Bhola', 'Barguna', 'Patuakhali');
```

প্রশ্ন 2: BETWEEN ব্যবহার করে যেসব পণ্যের স্টক ২০ থেকে ৫০-এর মধ্যে রয়েছে তাদের নাম ও স্টকের পরিমাণ প্রদর্শন করার কুয়েরি লিখুন।

```
SELECT product_name, stock_qty
FROM products
WHERE stock_qty BETWEEN 20 AND 50;
```

প্রশ্ন 3: LIKE অপারেটর দিয়ে যেসব গ্রাহকের নামের শুরুতে 'Md.' রয়েছে তাদের তালিকা ফিল্টার করার কুয়েরি লিখুন।

```
SELECT *
FROM customers
WHERE full_name LIKE 'Md.%';
```

প্রশ্ন 4: orders টেবিলের সকল অর্ডারকে মূল্যের ভিত্তিতে সর্বোচ্চ থেকে সর্বনিম্ন ক্রমানুসারে সাজানোর (ORDER BY total_amount DESC) কুয়েরি লিখুন।

```
SELECT *
FROM orders
ORDER BY total_amount DESC;
```

প্রশ্ন 5: সর্বাধিক মূল্যের শীর্ষ ৩টি অর্ডারের বিবরণ দেখতে ORDER BY এবং LIMIT সমন্বয় করে কুয়েরি লিখুন।

```
SELECT *
FROM orders
ORDER BY total_amount DESC
LIMIT 3;
```

মডিউল 05: টেবিল তৈরি ও ডেটা পরিবর্তন (DDL, DML)

প্রশ্ন 1: student_id (INT PRIMARY KEY), student_name (VARCHAR(100) NOT NULL) এবং department (VARCHAR(50)) কলাম নিয়ে students নামে একটি নতুন টেবিল তৈরির CREATE TABLE স্টেটমেন্ট লিখুন।

```
CREATE TABLE students (
  student_id INT PRIMARY KEY,
  student_name VARCHAR(100) NOT NULL,
  department VARCHAR(50)
);
```

প্রশ্ন 2: তৈরি করা students টেবিলে একসাথে ২টি নতুন শিক্ষার্থীর তথ্য যুক্ত করার জন্য INSERT INTO স্টেটমেন্ট লিখুন।

```
INSERT INTO students (student_id, student_name, department) VALUES
  (1, 'Md. Rafijur Rahman', 'Accounting'),
  (2, 'Saiful Islam Nuhash', 'Management');
```

প্রশ্ন 3: ১ নম্বর আইডিধারী শিক্ষার্থীর ডিপার্টমেন্টের নাম পরিবর্তন করে 'Finance' করার জন্য UPDATE ... WHERE কুয়েরি লিখুন।

```
UPDATE students
SET department = 'Finance'
WHERE student_id = 1;
```

প্রশ্ন 4: ২ নম্বর আইডিধারী শিক্ষার্থীর রেকর্ড মুছে ফেলার জন্য নিরাপদ DELETE ... WHERE কুয়েরি লিখুন।

```
DELETE FROM students
WHERE student_id = 2;
```

প্রশ্ন 5: তৈরি করা students টেবিলটি সম্পূর্ণ ডেটা ও কাঠামোসহ ডেটাবেস থেকে অপসারণের জন্য DROP TABLE স্টেটমেন্ট লিখুন।

```
DROP TABLE students;
```

মডিউল 06: Aggregate Functions ও GROUP BY

প্রশ্ন 1: orders টেবিল থেকে মোট অর্ডারের সংখ্যা (COUNT), মোট বিক্রির পরিমাণ (SUM) এবং গড় বিক্রয় মূল্য (AVG) নির্ণয় করার কুয়েরি লিখুন।

```
SELECT
  COUNT(*) AS total_orders,
  SUM(total_amount) AS total_sales,
  AVG(total_amount) AS avg_sales
FROM orders;
```

প্রশ্ন 2: orders টেবিল থেকে সর্বনিম্ন (MIN) এবং সর্বোচ্চ (MAX) অর্ডার মূল্য বের করার কুয়েরি লিখুন।

```
SELECT
  MIN(total_amount) AS min_amount,
  MAX(total_amount) AS max_amount
FROM orders;
```

প্রশ্ন 3: GROUP BY ব্যবহার করে প্রতিটি জেলার (city) মোট গ্রাহক সংখ্যা গণনা করার কুয়েরি লিখুন।

```
SELECT city, COUNT(*) AS customer_count
FROM customers
GROUP BY city;
```

প্রশ্ন 4: প্রতিটি গ্রাহকের (customer_id) মোট কত টাকার কেনাকাটা হয়েছে তা বের করতে GROUP BY এবং SUM ব্যবহার করে কুয়েরি লিখুন।

```
SELECT customer_id, SUM(total_amount) AS total_spent
FROM orders
GROUP BY customer_id;
```

প্রশ্ন 5: প্রতিটি ক্যাটাগরির পণ্যের গড় মূল্য হিসাব করে সর্বোচ্চ গড় মূল্যের ক্যাটাগরিকে সবার উপরে সাজানোর কুয়েরি লিখুন।

```
SELECT category, AVG(price) AS avg_price
FROM products
GROUP BY category
ORDER BY avg_price DESC;
```

মডিউল 07: HAVING ও SQL Execution Order

প্রশ্ন 1: orders টেবিল থেকে যেসব গ্রাহক সর্বমোট ২,০০০ টাকার বেশি কেনাকাটা করেছেন, HAVING ব্যবহার করে তাদের customer_id ও মোট টাকার পরিমাণ বের করার কুয়েরি লিখুন।

```
SELECT customer_id, SUM(total_amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(total_amount) > 2000;
```

প্রশ্ন 2: customers টেবিল থেকে যেসব জেলায় কমপক্ষে ২ জন সক্রিয় (status = 'Active') গ্রাহক রয়েছেন, WHERE, GROUP BY এবং HAVING সমন্বয় করে তাদের তালিকা বের করার কুয়েরি লিখুন।

```
SELECT city, COUNT(*) AS active_count
FROM customers
WHERE status = 'Active'
GROUP BY city
HAVING COUNT(*) >= 2;
```

প্রশ্ন 3: যেসব জেলায় মোট গ্রাহক সংখ্যা ১ জনের বেশি, HAVING ক্লজ দিয়ে শুধুমাত্র সেই জেলাগুলোর নাম ও গ্রাহক সংখ্যা প্রদর্শন করার কুয়েরি লিখুন।

```
SELECT city, COUNT(*) AS total_customers
FROM customers
GROUP BY city
HAVING COUNT(*) > 1;
```

প্রশ্ন 4: products টেবিল থেকে প্রতিটি ক্যাটাগরির পণ্যের মোট স্টক হিসাব করে যেসব ক্যাটাগরির মোট স্টক ১০০-এর কম তাদের ফিল্টার করার কুয়েরি লিখুন।

```
SELECT category, SUM(stock_qty) AS total_stock
FROM products
GROUP BY category
HAVING SUM(stock_qty) < 100;
```

প্রশ্ন 5: orders টেবিল থেকে যেসব গ্রাহকের মোট অর্ডারের সংখ্যা ১-এর বেশি (COUNT(order_id) > 1), তাদের আইডি ও অর্ডার সংখ্যা বের করার কুয়েরি লিখুন।

```
SELECT customer_id, COUNT(order_id) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(order_id) > 1;
```

মডিউল 08: CASE, COALESCE, NULLIF ও CAST

প্রশ্ন 1: CASE স্টেটমেন্ট ব্যবহার করে অর্ডারের টাকার পরিমাণ ৩,০০০ টাকার বেশি হলে 'High Value', ১,৫০০ থেকে ৩,০০০ হলে 'Medium Value', এবং বাকিগুলোকে 'Regular' লেবেল দিয়ে প্রদর্শন করার কুয়েরি লিখুন।

```
SELECT order_id, total_amount,
  CASE
    WHEN total_amount > 3000 THEN 'High Value'
    WHEN total_amount >= 1500 THEN 'Medium Value'
    ELSE 'Regular'
  END AS order_tier
FROM orders;
```

প্রশ্ন 2: COALESCE ব্যবহার করে গ্রাহকদের নামের পাশে তাদের ফোন নম্বর প্রদর্শন করুন; যদি ফোন নম্বর না থাকে তবে 'N/A' দেখানোর কুয়েরি লিখুন।

```
SELECT full_name, COALESCE(phone, 'N/A') AS contact_phone
FROM customers;
```

প্রশ্ন 3: NULLIF ফাংশন ব্যবহার করে অর্ডারের ডিসকাউন্ট ০ টাকা হলে তাকে NULL হিসেবে রূপান্তর করার কুয়েরি লিখুন।

```
SELECT order_id, discount, NULLIF(discount, 0) AS nullified_discount
FROM orders;
```

প্রশ্ন 4: CAST ফাংশন ব্যবহার করে পণ্যের ডেসিমাল বা ভগ্নাংশ মূল্যকে পূর্ণসংখ্যায় (INTEGER) রূপান্তর করার কুয়েরি লিখুন।

```
SELECT product_name, price, CAST(price AS INT) AS int_price
FROM products;
```

প্রশ্ন 5: CASE স্টেটমেন্ট দিয়ে গ্রাহকদের স্ট্যাটাস 'Active' হলে 'সক্রিয়' এবং অন্যথায় 'নিষ্ক্রিয়' হিসেবে বাংলায় রূপান্তর করে দেখানোর কুয়েরি লিখুন।

```
SELECT full_name,
  CASE status
    WHEN 'Active' THEN 'সক্রিয়'
    ELSE 'নিষ্ক্রিয়'
  END AS bangla_status
FROM customers;
```

মডিউল 09: INNER JOIN দিয়ে একাধিক টেবিল যুক্ত করা

প্রশ্ন 1: customers এবং orders টেবিল INNER JOIN করে প্রতিটি অর্ডারের সাথে সংশ্লিষ্ট গ্রাহকের নাম, অর্ডার আইডি এবং টাকার পরিমাণ প্রদর্শন করার কুয়েরি লিখুন।

```
SELECT c.full_name, o.order_id, o.total_amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
```

প্রশ্ন 2: INNER JOIN এবং WHERE সমন্বয় করে শুধুমাত্র বরিশাল (Barishal) জেলার গ্রাহকদের সম্পন্ন করা অর্ডারের তালিকা ফিল্টার করার কুয়েরি লিখুন।

```
SELECT c.full_name, c.city, o.order_id, o.total_amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE c.city = 'Barishal';
```

প্রশ্ন 3: customers, orders এবং order_items—এই ৩টি টেবিল একসাথে INNER JOIN করে প্রতিটি অর্ডারের আইটেম ও গ্রাহকের নাম প্রদর্শনের কুয়েরি লিখুন।

```
SELECT c.full_name, o.order_id, oi.product_id, oi.quantity, oi.unit_price
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
INNER JOIN order_items oi ON o.order_id = oi.order_id;
```

প্রশ্ন 4: customers এবং orders টেবিল INNER JOIN ও GROUP BY করে জেলাভিত্তিক মোট বিক্রির রাজস্ব (SUM(total_amount)) বের করার কুয়েরি লিখুন।

```
SELECT c.city, SUM(o.total_amount) AS total_revenue
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.city;
```

প্রশ্ন 5: orders এবং order_items টেবিল INNER JOIN করে প্রতিটি অর্ডারে মোট কয়টি আইটেম বিক্রি হয়েছে তা বের করার কুয়েরি লিখুন।

```
SELECT o.order_id, SUM(oi.quantity) AS total_items
FROM orders o
INNER JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.order_id;
```

মডিউল 10: LEFT JOIN ও Outer Join

প্রশ্ন 1: LEFT JOIN ব্যবহার করে সব গ্রাহকের তালিকা এবং তাদের অর্ডারের তথ্য আনুন, যাতে কোনো অর্ডার না করা গ্রাহকরাও বাদ না পড়েন।

```
SELECT c.customer_id, c.full_name, o.order_id, o.total_amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

প্রশ্ন 2: LEFT JOIN এবং COUNT(o.order_id) ব্যবহার করে প্রতিটি গ্রাহকের নাম এবং তাদের মোট সম্পন্ন করা সঠিক অর্ডার সংখ্যা গণনা করার কুয়েরি লিখুন।

```
SELECT c.full_name, COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.full_name;
```

প্রশ্ন 3: Anti Join প্যাটার্ন (LEFT JOIN ... WHERE o.order_id IS NULL) ব্যবহার করে যেসব গ্রাহক এখনো কোনো অর্ডার করেননি তাঁদের নাম ও জেলা বের করার কুয়েরি লিখুন।

```
SELECT c.customer_id, c.full_name, c.city
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

প্রশ্ন 4: LEFT JOIN ব্যবহার করে যেসব গ্রাহকের কোনো অর্ডার নেই তাদের মোট ব্যয়ের ঘরে COALESCE(SUM(o.total_amount), 0) দিয়ে ০ দেখানোর কুয়েরি লিখুন।

```
SELECT c.full_name, COALESCE(SUM(o.total_amount), 0) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.full_name;
```

প্রশ্ন 5: orders টেবিলকে বামে এবং customers টেবিলকে ডানে রেখে LEFT JOIN চালিয়ে প্রতিটি অর্ডারের সাথে গ্রাহকের নাম মেলানোর কুয়েরি লিখুন।

```
SELECT o.order_id, o.total_amount, c.full_name
FROM orders o
LEFT JOIN customers c ON o.customer_id = c.customer_id;
```

মডিউল 11: SELF JOIN ও CROSS JOIN

প্রশ্ন 1: employees টেবিলকে নিজের সাথেই INNER JOIN করে কোন কর্মকর্তার ঊর্ধ্বতন ম্যানেজার কে, তা পাশাপাশি প্রদর্শনের কুয়েরি লিখুন।

```
SELECT e.full_name AS employee, m.full_name AS manager
FROM employees e
INNER JOIN employees m ON e.manager_id = m.emp_id;
```

প্রশ্ন 2: employees টেবিলকে নিজের সাথে LEFT JOIN করে শীর্ষ প্রধান কর্মকর্তাসহ (যাঁর ম্যানেজার নেই) সকল কর্মীর নাম ও ম্যানেজারের নাম (না থাকলে 'None') প্রদর্শনের কুয়েরি লিখুন।

```
SELECT e.full_name AS employee, COALESCE(m.full_name, 'None') AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

প্রশ্ন 3: products এবং sizes টেবিলের মধ্যে CROSS JOIN চালিয়ে প্রতিটি পণ্যের সম্ভাব্য সকল সাইজের কম্বিনেশন তৈরি করার কুয়েরি লিখুন।

```
SELECT p.product_name, s.size_code
FROM products p
CROSS JOIN sizes s;
```

প্রশ্ন 4: employees টেবিল থেকে একই ম্যানেজারের অধীনে কর্মরত সহকর্মীদের (peers) জোড়া বের করতে SELF JOIN কুয়েরি লিখুন।

```
SELECT e1.full_name AS employee_1, e2.full_name AS employee_2, e1.manager_id
FROM employees e1
INNER JOIN employees e2 ON e1.manager_id = e2.manager_id AND e1.emp_id < e2.emp_id;
```

প্রশ্ন 5: employees টেবিলে কার অধীনে মোট কতজন কর্মী কর্মরত আছেন তা বের করতে SELF JOIN ও GROUP BY কুয়েরি লিখুন।

```
SELECT m.full_name AS manager, COUNT(e.emp_id) AS direct_reports
FROM employees m
INNER JOIN employees e ON m.emp_id = e.manager_id
GROUP BY m.emp_id, m.full_name;
```

মডিউল 12: Set Operators: UNION, INTERSECT ও EXCEPT

প্রশ্ন 1: online_customers এবং store_customers উভয় টেবিল থেকে অনন্য গ্রাহকদের নাম ও জেলা একত্রিত করতে UNION ব্যবহার করে কুয়েরি লিখুন।

```
SELECT full_name, city FROM online_customers
UNION
SELECT full_name, city FROM store_customers
ORDER BY full_name;
```

প্রশ্ন 2: ডুপ্লিকেট না ছেঁটে সকল অনলাইন ও অফলাইন গ্রাহকের রেকর্ড হুবহু একত্রিত করতে UNION ALL ব্যবহার করে কুয়েরি লিখুন।

```
SELECT full_name, city FROM online_customers
UNION ALL
SELECT full_name, city FROM store_customers;
```

প্রশ্ন 3: INTERSECT অপারেটর ব্যবহার করে অনলাইন শপ এবং ফিজিক্যাল শোরুম উভয় স্থান থেকেই কেনাকাটা করা সাধারণ গ্রাহকদের তালিকা খোঁজার কুয়েরি লিখুন।

```
SELECT full_name, city FROM online_customers
INTERSECT
SELECT full_name, city FROM store_customers;
```

প্রশ্ন 4: EXCEPT অপারেটর ব্যবহার করে শুধুমাত্র অনলাইন শপ থেকে কেনাকাটা করেছেন কিন্তু ফিজিক্যাল স্টোরে কেনাকাটা করেননি এমন গ্রাহকদের তালিকা বের করার কুয়েরি লিখুন।

```
SELECT full_name, city FROM online_customers
EXCEPT
SELECT full_name, city FROM store_customers
ORDER BY full_name;
```

প্রশ্ন 5: store_customers থেকে online_customers-কে বাদ দিয়ে শুধুমাত্র ফিজিক্যাল স্টোরের একক গ্রাহকদের নাম ও জেলা বের করতে EXCEPT কুয়েরি লিখুন।

```
SELECT full_name, city FROM store_customers
EXCEPT
SELECT full_name, city FROM online_customers
ORDER BY full_name;
```

মডিউল 13: String ও Numeric Functions

প্রশ্ন 1: TRIM() এবং UPPER() ফাংশন ব্যবহার করে গ্রাহকের নামের শুরুর ও শেষের বাড়তি স্পেস মুছে বড় হাতের অক্ষরে প্রদর্শনের কুয়েরি লিখুন।

```
SELECT TRIM(raw_name) AS clean_name, UPPER(TRIM(raw_name)) AS upper_name
FROM raw_customers;
```

প্রশ্ন 2: SUBSTR() ফাংশন ব্যবহার করে গ্রাহকের ফোন নম্বর থেকে প্রথম ৩ ডিজিট (অপারেটর কোড, যেমন 017) আলাদা করার কুয়েরি লিখুন।

```
SELECT TRIM(raw_name) AS name, SUBSTR(COALESCE(phone, '000'), 1, 3) AS operator_code
FROM raw_customers;
```

প্রশ্ন 3: LENGTH() ফাংশন ব্যবহার করে প্রতিটি গ্রাহকের পরিচ্ছন্ন নামের মোট অক্ষরের দৈর্ঘ্য গণনা করার কুয়েরি লিখুন।

```
SELECT TRIM(raw_name) AS clean_name, LENGTH(TRIM(raw_name)) AS name_length
FROM raw_customers;
```

প্রশ্ন 4: ABS() ফাংশন ব্যবহার করে গ্রাহকের ঋণাত্মক ব্যালেন্সকে পরম মান (ধনাত্মক সংখ্যা) হিসেবে রূপান্তর করার কুয়েরি লিখুন।

```
SELECT TRIM(raw_name) AS name, balance, ABS(balance) AS absolute_balance
FROM raw_customers;
```

প্রশ্ন 5: মডিউলো (% 100) অপারেটর ব্যবহার করে পরম ব্যালেন্সকে ১০০ টাকার নোটে রূপান্তরের পর অবশিষ্ট খুচরা টাকার পরিমাণ বের করার কুয়েরি লিখুন।

```
SELECT TRIM(raw_name) AS name, ABS(balance) % 100 AS rem_amount
FROM raw_customers;
```

মডিউল 14: Date & Time Functions

প্রশ্ন 1: strftime('%Y-%m', order_date) ব্যবহার করে অর্ডারের তারিখ থেকে বছর ও মাস আলাদা করে প্রদর্শনের কুয়েরি লিখুন।

```
SELECT order_id, order_date, strftime('%Y-%m', order_date) AS order_month
FROM orders;
```

প্রশ্ন 2: মাসিক বিক্রির রিপোর্ট তৈরিতে GROUP BY strftime('%Y-%m', order_date) ব্যবহার করে প্রতি মাসের মোট অর্ডার সংখ্যা ও মোট আয় বের করার কুয়েরি লিখুন।

```
SELECT strftime('%Y-%m', order_date) AS month, COUNT(order_id) AS total_orders, SUM(amount) AS
total_sales
FROM orders
GROUP BY strftime('%Y-%m', order_date)
ORDER BY month;
```

প্রশ্ন 3: date(order_date, '+30 days') মডিফায়ার ব্যবহার করে প্রতিটি অর্ডারের জন্য ৩০ দিন পরের বিল পরিশোধের শেষ তারিখ (due_date) হিসাব করার কুয়েরি লিখুন।

```
SELECT order_id, order_date, date(order_date, '+30 days') AS due_date
FROM orders;
```

প্রশ্ন 4: julianday('2026-03-31') - julianday(order_date) ব্যবহার করে নির্দিষ্ট তারিখ থেকে অর্ডারের দিনের ব্যবধান (days_ago) পূর্ণসংখ্যায় রূপান্তর করে দেখানোর কুয়েরি লিখুন।

```
SELECT order_id, order_date, CAST(julianday('2026-03-31') - julianday(order_date) AS INT) AS
days_ago
FROM orders;
```

প্রশ্ন 5: orders টেবিল থেকে গত ৩০ দিনের পুরনো (days_ago > 30) অর্ডারগুলো ফিল্টার করার কুয়েরি লিখুন।

```
SELECT order_id, order_date, amount
FROM orders
WHERE CAST(julianday('2026-03-31') - julianday(order_date) AS INT) > 30;
```

মডিউল 15: Subqueries ও EXISTS

প্রশ্ন 1: Scalar Subquery ব্যবহার করে গড় অর্ডার মূল্যের (AVG(amount)) চেয়ে বেশি মূল্যের অর্ডারগুলো ফিল্টার করার কুয়েরি লিখুন।

```
SELECT order_id, customer_id, amount
FROM orders
WHERE amount > (SELECT AVG(amount) FROM orders)
ORDER BY amount DESC;
```

প্রশ্ন 2: IN সাবকুয়েরি ব্যবহার করে যেসব গ্রাহক ২,০০০ টাকার বেশি মূল্যের অর্ডার দিয়েছেন তাদের নাম ও জেলা বের করার কুয়েরি লিখুন।

```
SELECT customer_id, full_name, city
FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders WHERE amount >= 2000)
ORDER BY customer_id;
```

প্রশ্ন 3: Correlated Subquery এবং EXISTS ব্যবহার করে অন্তত একটি অর্ডার সম্পন্ন করা গ্রাহকদের তালিকা বের করার কুয়েরি লিখুন।

```
SELECT c.customer_id, c.full_name
FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY c.customer_id;
```

প্রশ্ন 4: NOT EXISTS ব্যবহার করে যেসব গ্রাহকের কোনো অর্ডারের অস্তিত্ব নেই তাঁদের নাম ও জেলা ফিল্টার করার কুয়েরি লিখুন।

```
SELECT c.customer_id, c.full_name, c.city
FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

প্রশ্ন 5: FROM ক্লজের ভেতরে সাবকুয়েরি (Derived Table) ব্যবহার করে গ্রাহকপ্রতি খরচের ওপর সামগ্রিক গড় ব্যয় হিসাব করার কুয়েরি লিখুন।

```
SELECT AVG(cust_total) AS overall_avg_spend
FROM (
  SELECT customer_id, SUM(amount) AS cust_total
  FROM orders
  GROUP BY customer_id
);
```

মডিউল 16: Common Table Expressions (CTE)

প্রশ্ন 1: WITH ক্লজ ব্যবহার করে একটি সাধারণ CTE (order_stats) তৈরি করুন যা অর্ডারের গড় মান বের করবে এবং মূল কুয়েরিতে গড়ের চেয়ে বড় অর্ডারগুলো ফিল্টার করবে।

```
WITH order_stats AS (
  SELECT AVG(amount) AS avg_amt FROM orders
)
SELECT o.customer_name, o.amount
FROM orders o, order_stats s
WHERE o.amount > s.avg_amt
ORDER BY o.amount DESC;
```

প্রশ্ন 2: Chained CTE তৈরি করে প্রথমে ১,৫০০ টাকার সমান বা বেশি মূল্যের অর্ডার আলাদা করুন এবং পরবর্তী CTE-তে তার ওপর দ্বিগুণ বোনাস পয়েন্ট হিসাব করার কুয়েরি লিখুন।

```
WITH high_orders AS (
  SELECT customer_name, amount FROM orders WHERE amount >= 1500
),
bonus AS (
  SELECT customer_name, amount, amount * 2 AS points FROM high_orders
)
SELECT customer_name, amount, points FROM bonus ORDER BY amount DESC;
```

প্রশ্ন 3: WITH RECURSIVE ব্যবহার করে ১ থেকে ৫ পর্যন্ত ক্রমিক সংখ্যার একটি ধারা তৈরির কুয়েরি লিখুন।

```
WITH RECURSIVE seq(num) AS (
  SELECT 1
  UNION ALL
  SELECT num + 1 FROM seq WHERE num < 5
)
SELECT num FROM seq;
```

প্রশ্ন 4: WITH RECURSIVE ব্যবহার করে '2026-03-01' থেকে শুরু করে পরপর ৪ দিনের তারিখের তালিকা তৈরি করার কুয়েরি লিখুন।

```
WITH RECURSIVE dates(d) AS (
  SELECT '2026-03-01'
  UNION ALL
  SELECT date(d, '+1 day') FROM dates WHERE d < '2026-03-04'
)
SELECT d FROM dates;
```

প্রশ্ন 5: CTE ব্যবহার করে orders টেবিলের সর্বোচ্চ অর্ডার মূল্য (MAX(amount)) বের করে সেই সর্বোচ্চ অর্ডারের বিবরণ প্রদর্শনের কুয়েরি লিখুন।

```
WITH max_val AS (
  SELECT MAX(amount) AS m FROM orders
)
SELECT o.* FROM orders o, max_val v WHERE o.amount = v.m;
```

মডিউল 17: Window Functions (পার্ট ১): OVER ও Ranking

প্রশ্ন 1: ROW_NUMBER() OVER (ORDER BY amount DESC) ব্যবহার করে সকল অর্ডারকে মূল্যের ভিত্তিতে ক্রমিক নম্বর দেওয়ার কুয়েরি লিখুন।

```
SELECT order_id, customer_name, amount,
  ROW_NUMBER() OVER (ORDER BY amount DESC) AS row_num
FROM orders;
```

প্রশ্ন 2: DENSE_RANK() OVER (ORDER BY amount DESC) ব্যবহার করে অর্ডারের মূল্যের ওপর ভিত্তি করে কোনো সংখ্যা বাদ না দিয়ে র‍্যাংকিং তৈরি করার কুয়েরি লিখুন।

```
SELECT order_id, customer_name, amount,
  DENSE_RANK() OVER (ORDER BY amount DESC) AS dense_rnk
FROM orders;
```

প্রশ্ন 3: PARTITION BY city ব্যবহার করে প্রতিটি জেলার অর্ডারের ওপর আলাদা আলাদা র‍্যাংক (city_rnk) তৈরির কুয়েরি লিখুন।

```
SELECT city, order_id, amount,
  DENSE_RANK() OVER (PARTITION BY city ORDER BY amount DESC) AS city_rnk
FROM orders;
```

প্রশ্ন 4: কোনো সারি গ্রুপ না করে প্রতিটি অর্ডারের পাশে সমগ্র টেবিলের গড় মূল্য AVG(amount) OVER () প্রদর্শনের কুয়েরি লিখুন।

```
SELECT customer_name, amount,
  CAST(AVG(amount) OVER () AS INT) AS avg_amt
FROM orders
ORDER BY amount DESC;
```

প্রশ্ন 5: CTE এবং DENSE_RANK() OVER (PARTITION BY city ORDER BY amount DESC) ব্যবহার করে প্রতিটি জেলার সর্বোচ্চ কেনাকাটা করা শীর্ষ ১ নম্বর অর্ডারটি ফিল্টার করার কুয়েরি লিখুন।

```
WITH ranked_orders AS (
  SELECT *, DENSE_RANK() OVER (PARTITION BY city ORDER BY amount DESC) AS rk
  FROM orders
)
SELECT * FROM ranked_orders WHERE rk = 1;
```

মডিউল 18: Window Functions (পার্ট ২): LAG, LEAD ও ট্রেন্ড

প্রশ্ন 1: LAG(revenue) OVER (ORDER BY month_id) ব্যবহার করে প্রতিটি মাসের আয়ের পাশে তার ঠিক আগের মাসের রাজস্ব প্রদর্শনের কুয়েরি লিখুন।

```
SELECT month_id, revenue,
  LAG(revenue) OVER (ORDER BY month_id) AS prev_month_rev
FROM monthly_sales;
```

প্রশ্ন 2: revenue - LAG(revenue) OVER (ORDER BY month_id) হিসাব করে মাসভিত্তিক রাজস্ব প্রবৃদ্ধি (mom_growth) বের করার কুয়েরি লিখুন।

```
SELECT month_id, revenue,
  revenue - LAG(revenue) OVER (ORDER BY month_id) AS mom_growth
FROM monthly_sales;
```

প্রশ্ন 3: LEAD(revenue) OVER (ORDER BY month_id) ব্যবহার করে চলতি মাসের পাশে পরবর্তী মাসের সম্ভাব্য রাজস্ব প্রদর্শনের কুয়েরি লিখুন।

```
SELECT month_id, revenue,
  LEAD(revenue) OVER (ORDER BY month_id) AS next_month_rev
FROM monthly_sales;
```

প্রশ্ন 4: SUM(revenue) OVER (ORDER BY month_id) ব্যবহার করে মাসের পর মাস ক্রমবর্ধমান মোট রাজস্ব (Running Total) হিসাব করার কুয়েরি লিখুন।

```
SELECT month_id, revenue,
  SUM(revenue) OVER (ORDER BY month_id) AS running_total
FROM monthly_sales;
```

প্রশ্ন 5: AVG(revenue) OVER (ORDER BY month_id ROWS BETWEEN 1 PRECEDING AND CURRENT ROW) ব্যবহার করে বিগত ২ মাসের চলমান গড় (Moving Average) হিসাব করার কুয়েরি লিখুন।

```
SELECT month_id, revenue,
  CAST(AVG(revenue) OVER (ORDER BY month_id ROWS BETWEEN 1 PRECEDING AND CURRENT ROW) AS INT)
AS moving_avg
FROM monthly_sales;
```

মডিউল 19: Indexing ও Data Modeling

প্রশ্ন 1: orders টেবিলের customer_id কলামের ওপর idx_orders_customer নামে একটি একক ইনডেক্স তৈরির CREATE INDEX স্টেটমেন্ট লিখুন।

```
CREATE INDEX idx_orders_customer ON orders(customer_id);
```

প্রশ্ন 2: orders টেবিলের customer_id এবং order_date কলাম দুটির ওপর idx_orders_cust_date নামে একটি কম্পোজিট ইনডেক্স তৈরির স্টেটমেন্ট লিখুন।

```
CREATE INDEX idx_orders_cust_date ON orders(customer_id, order_date);
```

প্রশ্ন 3: EXPLAIN QUERY PLAN ব্যবহার করে SELECT * FROM orders WHERE customer_id = 1; কুয়েরিটি ইনডেক্স ব্যবহার করছে কিনা তা নিরীক্ষার SQL লিখুন।

```
EXPLAIN QUERY PLAN
SELECT * FROM orders WHERE customer_id = 1;
```

প্রশ্ন 4: ডেটাবেস থেকে পূর্বের তৈরি করা idx_orders_customer ইনডেক্সটি ড্রপ বা মুছে ফেলার জন্য DROP INDEX স্টেটমেন্টটি লিখুন।

```
DROP INDEX idx_orders_customer;
```

প্রশ্ন 5: customers টেবিলের phone কলামের ওপর ডুপ্লিকেট ফোন নম্বর প্রতিরোধ করতে idx_customers_phone_unique নামে একটি ইউনিক ইনডেক্স তৈরির স্টেটমেন্ট লিখুন।

```
CREATE UNIQUE INDEX idx_customers_phone_unique ON customers(phone);
```

মডিউল 20: BD Shop ক্যাপস্টোন প্রজেক্ট

প্রশ্ন 1: customers এবং orders টেবিল যুক্ত করে বরিশাল বিভাগের কোন জেলায় সবচেয়ে বেশি রাজস্ব এসেছে তা নির্ণয়ের কুয়েরি লিখুন।

```
SELECT c.city, SUM(o.total_amount) AS total_revenue
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.city
ORDER BY total_revenue DESC;
```

প্রশ্ন 2: categories, products ও order_items টেবিল জয়েন করে সর্বাধিক বিক্রিত পণ্য ক্যাটাগরি শনাক্ত করার কোড লিখুন।

```
SELECT cat.category_name, SUM(oi.quantity) AS total_sold
FROM categories cat
INNER JOIN products p ON cat.category_id = p.category_id
INNER JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY cat.category_name
ORDER BY total_sold DESC;
```

প্রশ্ন 3: Anti Join ব্যবহার করে যেসকল নিবন্ধিত গ্রাহক কখনো কোনো অর্ডার করেননি তাঁদের নাম ও জেলার তালিকা বের করুন।

```
SELECT c.customer_id, c.full_name, c.city
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

প্রশ্ন 4: CTE এবং DENSE_RANK() OVER (PARTITION BY city ORDER BY total_spend DESC) ব্যবহার করে প্রতিটি জেলার সর্বোচ্চ কেনাকাটা করা শীর্ষ ১ জন ক্রেতাকে ফিল্টার করার কুয়েরি লিখুন।

```
WITH customer_spending AS (
  SELECT c.customer_id, c.full_name, c.city, SUM(o.total_amount) AS total_spend
  FROM customers c
  INNER JOIN orders o ON c.customer_id = o.customer_id
  GROUP BY c.customer_id, c.full_name, c.city
),
ranked_customers AS (
  SELECT *, DENSE_RANK() OVER (PARTITION BY city ORDER BY total_spend DESC) AS rk
  FROM customer_spending
)
SELECT customer_id, full_name, city, total_spend
FROM ranked_customers
WHERE rk = 1;
```

প্রশ্ন 5: orders টেবিল থেকে strftime('%Y-%m', order_date) ব্যবহার করে প্রতি মাসের মোট অর্ডার সংখ্যা ও মোট রাজস্ব পরিমাপের মাসিক সেলস ট্রেন্ড কুয়েরি লিখুন।

```
SELECT strftime('%Y-%m', order_date) AS sales_month,
  COUNT(order_id) AS total_orders,
  SUM(total_amount) AS monthly_revenue
FROM orders
GROUP BY sales_month
ORDER BY sales_month ASC;
```